

Product Requirements Document (PRD)

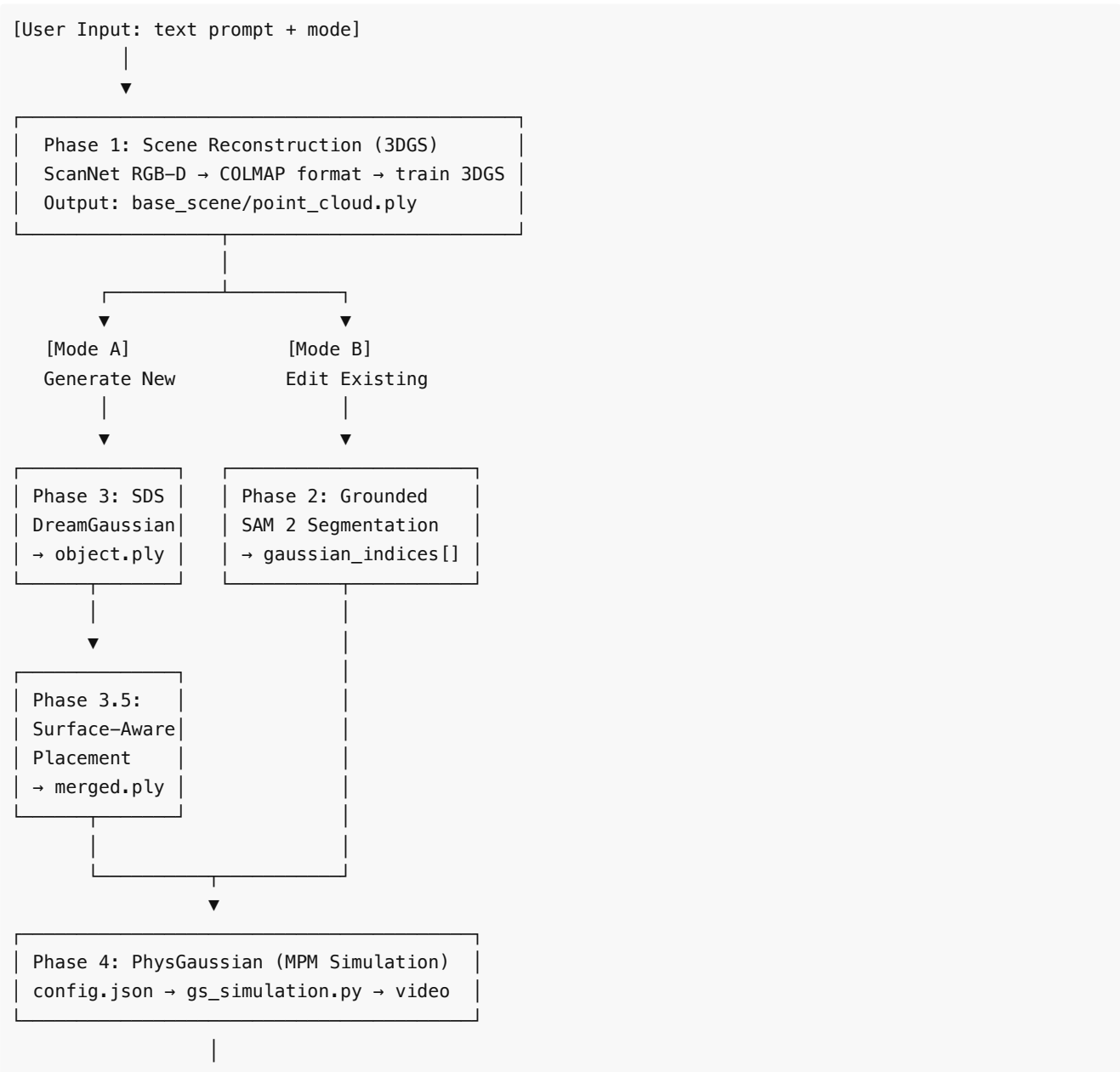
Project: Text Guided 3D Room Editor with Interactive Effects **Author:** Minjae Bang (Columbia University, SEAS) **Target Reader:** Cursor AI (AI Coding Assistant) **Purpose:** This document provides a complete, step-by-step implementation blueprint for building a text-guided 3D indoor scene editing system. Cursor AI should follow each phase sequentially, completing all tasks and passing the validation criteria before advancing to the next phase.

0. Executive Summary

This project builds a unified pipeline that allows a user to (1) generate a brand-new 3D object from a text prompt and insert it into an existing 3D Gaussian Splatting (3DGS) scene, and (2) select an existing object in the scene via text and apply physics-based effects such as melting, inflation, or jelly-like wobble. The pipeline integrates four major open-source components: 3D Gaussian Splatting for scene representation, Grounded SAM 2 for text-driven segmentation, DreamGaussian for SDS-based object generation, and PhysGaussian for MPM-based physics simulation.

1. System Architecture

1.1 High-Level Pipeline Flow



▼
[Output: .mp4 video]

1.2 Tech Stack

Component	Repository	Role	Key Files
3DGS	graphdeco-inria/gaussian-splatting	Scene reconstruction & rendering	train.py, render.py
Grounded SAM 2	IDEA-Research/Grounded-SAM-2	Text → bounding box → segmentation mask	grounded_sam2_local_demo.py
DreamGaussian	dreamgaussian/dreamgaussian	Text → 3D Gaussian object via SDS	main.py, guidance/sd_utils.py
PhysGaussian	XPandora/PhysGaussian	MPM physics simulation on Gaussians	gs_simulation.py, mpm_solver_warp/
Stable Diffusion	HuggingFace (stabilityai/stable-diffusion-2-1-base)	2D diffusion prior for SDS loss	Loaded via diffusers

1.3 Target Directory Structure

```
text-guided-3d-editor/
├── setup.sh                # One-click environment setup
├── requirements.txt
├── README.md
├──
├── data/
│   ├── scannet_raw/       # Raw ScanNet scene (downloaded separately)
│   │   ├── scene0000_00/
│   │   │   ├── color/    # RGB images
│   │   │   ├── depth/    # Depth maps
│   │   │   ├── pose/     # Camera extrinsics (4x4 .txt)
│   │   │   └── intrinsic/ # Camera intrinsics
│   └── scannet_colmap/    # Converted COLMAP format (generated)
│       ├── scene0000_00/
│       │   ├── images/
│       │   └── sparse/0/
│       │       ├── cameras.bin
│       │       ├── images.bin
│       └── points3D.bin
├── submodules/           # Git submodules (DO NOT modify internals)
│   ├── gaussian-splatting/
│   ├── Grounded-SAM-2/
│   ├── dreamgaussian/
│   └── PhysGaussian/
├──
├── output/
│   ├── base_scene/       # Trained 3DGS model
│   │   └── point_cloud/iteration_30000/point_cloud.ply
│   ├── generated_objects/ # SDS-generated objects
│   ├── merged_scenes/    # After placement
│   ├── sim_results/      # PhysGaussian output frames & video
│   └── eval/             # Quantitative evaluation results
```

```

├── src/
│   ├── __init__.py
│   ├── pipeline.py          # Main orchestrator
│   ├── config.py           # Pydantic config model
│   │
│   ├── data_utils/
│   │   ├── __init__.py
│   │   ├── scannet_to_colmap.py  # ScanNet → COLMAP converter
│   │   └── scannet_downloader.py # Helper to download ScanNet scenes
│   │
│   ├── reconstruction/
│   │   ├── __init__.py
│   │   ├── train_3dgs.py        # Wrapper for 3DGS training
│   │   └── render_views.py     # Render images from trained 3DGS
│   │
│   ├── segmentation/
│   │   ├── __init__.py
│   │   ├── text_to_mask.py      # Grounding DINO + SAM 2 wrapper
│   │   ├── mask_to_gaussians.py # 2D mask → 3D Gaussian indices
│   │   └── multi_view_consensus.py # Multi-view mask voting for robustness
│   │
│   ├── generation/
│   │   ├── __init__.py
│   │   ├── sds_generator.py     # DreamGaussian wrapper
│   │   └── object_rescaler.py   # Scale generated object to scene units
│   │
│   ├── placement/
│   │   ├── __init__.py
│   │   ├── surface_aware.py     # Depth-based surface detection
│   │   └── ply_merger.py        # Merge two PLY files with SH alignment
│   │
│   ├── physics/
│   │   ├── __init__.py
│   │   ├── config_generator.py  # Generate PhysGaussian JSON config
│   │   ├── mpm_runner.py        # Execute gs_simulation.py
│   │   └── material_presets.py  # Predefined material parameters
│   │
│   └── evaluation/
│       ├── __init__.py
│       ├── metrics.py           # PSNR, SSIM, LPIPS computation
│       └── ablation.py          # SAM 2 vs LERF comparison
│
├── configs/
│   ├── pipeline_config.yaml    # Global pipeline configuration
│   └── phys_templates/
│       ├── jelly.json
│       ├── metal.json
│       ├── sand.json
│       └── foam.json
│
├── scripts/
│   ├── run_mode_a.sh          # End-to-end Mode A demo
│   ├── run_mode_b.sh          # End-to-end Mode B demo
│   └── run_evaluation.sh       # Run quantitative evaluation
│
└── tests/
    └── test_scannet_converter.py

```

```
|— test_segmentation.py
|— test_sds_generation.py
|— test_placement.py
└— test_physics.py
```

2. Implementation Phases

PHASE 1: Environment Setup & Scene Reconstruction

Goal: Set up the development environment, convert ScanNet data, and train a base 3DGS scene. **Duration:** Week 1-2

Task 1.1: Environment Setup Script (`setup.sh`)

Write a bash script that automates the entire environment setup. The script must handle the following in order:

```
#!/bin/bash
set -e

# 1. Create conda environment
conda create -n room-editor python=3.10 -y
conda activate room-editor

# 2. Install PyTorch with CUDA
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 --index-url
https://download.pytorch.org/whl/cu118

# 3. Clone submodules
git submodule update --init --recursive

# 4. Install 3DGS dependencies
pip install -e submodules/gaussian-splatting/submodules/diff-gaussian-rasterization/
pip install -e submodules/gaussian-splatting/submodules/simple-knn/

# 5. Install Grounded SAM 2
cd submodules/Grounded-SAM-2
pip install -e .
pip install --no-build-isolation -e grounding_dino
cd checkpoints && bash download_ckpts.sh && cd ..
cd gdino_checkpoints && bash download_ckpts.sh && cd ../../..

# 6. Install DreamGaussian dependencies
pip install -r submodules/dreamgaussian/requirements.txt

# 7. Install PhysGaussian dependencies
pip install -r submodules/PhysGaussian/requirements.txt

# 8. Install project-level dependencies
pip install plyfile trimesh lpips scikit-image pyyaml pydantic typer rich
```

Validation: Run `python -c "import torch; print(torch.cuda.is_available())"` and confirm it returns `True`. Run `python -c "from diff_gaussian_rasterization import GaussianRasterizer"` to confirm 3DGS compilation.

Task 1.2: ScanNet to COLMAP Converter (`src/data_utils/scannet_to_colmap.py`)

This script converts extracted ScanNet scene data into COLMAP binary format.

Input directory structure:

```

scannet_raw/scene0000_00/
├─ color/          # 0.jpg, 1.jpg, ...
├─ depth/          # 0.png, 1.png, ... (16-bit depth in mm)
├─ pose/           # 0.txt, 1.txt, ... (4x4 camera-to-world matrix)
└─ intrinsic/
    └─ intrinsic_depth.txt # 4x4 intrinsic matrix

```

Output directory structure:

```

scannet_colmap/scene0000_00/
├─ images/          # Symlinked or copied RGB images
└─ sparse/0/
    ├─ cameras.bin  # Single PINHOLE camera model
    ├─ images.bin   # Per-image quaternion + translation
    └─ points3D.bin # Initial sparse point cloud from depth backprojection

```

Key implementation details:

1. Read the 4x4 intrinsic matrix and extract `fx`, `fy`, `cx`, `cy` for the COLMAP `PINHOLE` camera model.
2. For each pose file, read the 4x4 camera-to-world matrix, invert it to get world-to-camera, then decompose into quaternion (`w`, `x`, `y`, `z`) and translation (`tx`, `ty`, `tz`) in COLMAP convention.
3. Generate an initial sparse point cloud by backprojecting every Nth depth pixel (e.g., every 20th pixel) from a subset of frames (e.g., every 10th frame) using the corresponding pose. This gives 3DGS a good initialization.
4. Use the `read_write_model` utilities from COLMAP's Python scripts or the `pycolmap` library to write binary files.

Validation: Load the output with `pycolmap` or the 3DGS data loader and confirm no errors.

Task 1.3: 3DGS Training Wrapper (`src/reconstruction/train_3dgs.py`)

Write a Python wrapper that calls the 3DGS training script with appropriate arguments.

```

import subprocess
import os

def train_scene(data_path: str, output_path: str, iterations: int = 30000):
    """Train 3DGS on a COLMAP-formatted scene."""
    cmd = [
        "python", "submodules/gaussian-splatting/train.py",
        "-s", data_path,
        "-m", output_path,
        "--iterations", str(iterations),
    ]
    subprocess.run(cmd, check=True)

    # Verify output
    ply_path = os.path.join(output_path, "point_cloud", f"iteration_{iterations}",
                             "point_cloud.ply")
    assert os.path.exists(ply_path), f"Training failed: {ply_path} not found"
    return ply_path

```

Validation: The output `point_cloud.ply` should contain > 100,000 Gaussians for a typical ScanNet scene. Render a few views using `render.py` and visually inspect.

Task 1.4: View Renderer (`src/reconstruction/render_views.py`)

Write a utility to render RGB images and depth maps from the trained 3DGS model at any camera viewpoint. This will be used by the segmentation module.

Output per view:

- `rgb_{view_id}.png` (H x W x 3, uint8)
- `depth_{view_id}.npy` (H x W, float32, in scene units)
- `alpha_{view_id}.png` (H x W, uint8, opacity map)

PHASE 2: Semantic Segmentation with Grounded SAM 2

Goal: Given a text query and a camera view, identify which 3D Gaussians belong to the target object. **Duration:** Week 3

Task 2.1: Text-to-2D-Mask (`src/segmentation/text_to_mask.py`)

Wrap the Grounded SAM 2 pipeline into a reusable Python function.

```
def text_to_mask(image_path: str, text_prompt: str,
                 box_threshold: float = 0.3,
                 text_threshold: float = 0.25) -> np.ndarray:
    """
    Args:
        image_path: Path to the rendered RGB image.
        text_prompt: Object description (e.g., "the wooden chair").
        box_threshold: Confidence threshold for Grounding DINO detection.
        text_threshold: Text similarity threshold for Grounding DINO.

    Returns:
        mask: Binary mask (H, W) as np.ndarray of dtype bool.
    """
```

Implementation steps:

1. Load the Grounding DINO model (local checkpoint at `submodules/Grounded-SAM-2/gdino_checkpoints/groundingdino_swinb_cogcoor.pth`).
2. Run Grounding DINO with the text prompt to get bounding boxes.
3. Initialize the SAM 2 image predictor (checkpoint at `submodules/Grounded-SAM-2/checkpoints/sam2.1_hiera_large.pt`).
4. Pass the bounding boxes as box prompts to SAM 2.
5. Return the highest-confidence mask.

Memory management: After inference, explicitly delete the models and call `torch.cuda.empty_cache()` . These models will not be needed during Phase 3 and 4.

Task 2.2: 2D Mask to 3D Gaussian Indices (`src/segmentation/mask_to_gaussians.py`)

```
def mask_to_gaussian_indices(
    mask: np.ndarray,          # (H, W) bool
    depth_map: np.ndarray,     # (H, W) float32
    camera_intrinsics: np.ndarray, # (3, 3)
    camera_extrinsics: np.ndarray, # (4, 4) world-to-camera
    gaussian_positions: np.ndarray, # (N, 3) xyz of all Gaussians
    distance_threshold: float = 0.05
) -> np.ndarray:
    """
    Returns:
        indices: 1D array of integer indices into the Gaussian array.
    """
```

Implementation steps:

1. For each True pixel in the mask, backproject it to 3D using depth and camera parameters.
2. Build a KD-tree from `gaussian_positions`.
3. For each backprojected 3D point, query the KD-tree for the nearest Gaussian within `distance_threshold`.
4. Collect all unique Gaussian indices.

Task 2.3: Multi-View Consensus (`src/segmentation/multi_view_consensus.py`)

To improve robustness, run Task 2.1 and 2.2 from multiple camera viewpoints and take the intersection (or majority vote) of the resulting Gaussian index sets.

Input: List of camera view indices (e.g., `[0, 10, 20, 30]`). **Output:** Final set of Gaussian indices that appear in at least 2 out of N views.

PHASE 3: SDS-Based Object Generation & Placement

Goal: Generate a new 3D Gaussian object from text and place it in the scene. **Duration:** Week 4-5

Task 3.1: SDS Object Generator (`src/generation/sds_generator.py`)

Wrap DreamGaussian's text-to-3D pipeline.

```
def generate_object(text_prompt: str, output_dir: str,
                   num_steps: int = 500) -> str:
    """
    Args:
        text_prompt: Description of the object (e.g., "a red rubber duck").
        output_dir: Directory to save the generated .ply file.
        num_steps: Number of SDS optimization steps.

    Returns:
        ply_path: Path to the generated object PLY file.
    """
```

Key considerations:

1. DreamGaussian generates objects centered at the origin. The output PLY must be in the same coordinate system and scale as the base scene.
2. After generation, run `src/generation/object_rescaler.py` to normalize the object size relative to the scene (e.g., a "rubber duck" should be ~0.15m, not 2m).
3. If DreamGaussian's SDS quality is insufficient, consider using its two-stage approach: SDS optimization followed by mesh extraction and texture refinement.

Fallback strategy: If SDS optimization produces poor results (Janus artifacts, blurry textures), switch to using a pre-trained 3D generation model like `InstantMesh` or `TripoSr` to generate a mesh first, then convert the mesh to Gaussians using `gaussian-splatting`'s initialization.

Task 3.2: Surface-Aware Placement (`src/placement/surface_aware.py`)

```
def find_surface_height(scene_gaussians: np.ndarray,
                       x: float, y: float,
                       search_radius: float = 0.1) -> float:
    """
    Find the z-coordinate of the nearest surface at (x, y).

    Process:
    1. Filter Gaussians within search_radius of (x, y) in the xy-plane.
    2. Among filtered Gaussians, find the one with the highest z that has
       high opacity (> 0.5) - this represents the top of a surface.
```

```
3. Return that z value as the placement height.
"""
```

Task 3.3: PLY Merger (`src/placement/ply_merger.py`)

```
def merge_ply_files(base_ply: str, object_ply: str,
                    translation: np.ndarray, # (3,) offset
                    output_ply: str) -> str:
    """
    Merge two 3DGS PLY files.

    CRITICAL: Both PLY files must have identical vertex property schemas.
    3DGS PLY files contain: x, y, z, nx, ny, nz, f_dc_0..2, f_rest_0..44,
    opacity, scale_0..2, rot_0..3.

    Steps:
    1. Load both PLY files using plyfile.PlyData.
    2. Apply translation to the object's x, y, z coordinates.
    3. Verify both have the same property list.
    4. Concatenate vertex arrays.
    5. Save as new PLY file.
    """
```

PHASE 4: Physics Simulation with PhysGaussian

Goal: Apply MPM-based physics effects to selected or generated Gaussians. **Duration:** Week 6-7

Task 4.1: Material Presets (`src/physics/material_presets.py`)

Define a dictionary of material configurations matching PhysGaussian's expected parameters.

```
MATERIAL_PRESETS = {
    "jelly": {
        "material": "jelly",
        "E": 2e4,          # Young's modulus (Pa)
        "nu": 0.3,         # Poisson's ratio
        "density": 1000,   # kg/m^3
    },
    "metal": {
        "material": "metal",
        "E": 5e5,
        "nu": 0.35,
        "density": 7800,
    },
    "sand": {
        "material": "sand",
        "E": 1e4,
        "nu": 0.2,
        "density": 1500,
    },
    "foam": {
        "material": "foam",
        "E": 1e3,
        "nu": 0.25,
        "density": 200,
    },
}
```

```
    },  
}
```

Task 4.2: Config Generator (`src/physics/config_generator.py`)

```
def generate_phys_config(  
    gaussian_indices: np.ndarray,  
    gaussian_positions: np.ndarray,  
    material_name: str,  
    output_path: str,  
    n_grid: int = 100,  
    frame_num: int = 120,  
    frame_dt: float = 0.01,  
    substep_dt: float = 4e-4,  
    gravity: float = -9.8,  
    camera_index: int = 0  
) -> str:  
    """  
    Generate a PhysGaussian-compatible JSON config file.  
  
    The config must include:  
    - Preprocessing: opacity_threshold, rotation, sim_area  
    - Simulation: material params, grid, gravity, boundary conditions  
    - Export: frame count, camera index  
    """
```

Key detail: PhysGaussian's `sim_area` must be computed from the bounding box of the target Gaussians, expanded by a 20% margin. The `boundary_conditions` should include a `bounding_box` type to prevent particles from escaping the simulation domain.

Task 4.3: MPM Runner (`src/physics/mpm_runner.py`)

```
def run_simulation(model_path: str, config_path: str,  
                  output_path: str) -> str:  
    """  
    Execute PhysGaussian simulation and return path to output video.  
    """  
    cmd = [  
        "python", "submodules/PhysGaussian/gs_simulation.py",  
        "--model_path", model_path,  
        "--output_path", output_path,  
        "--config", config_path,  
        "--render_img",  
        "--compile_video",  
    ]  
    subprocess.run(cmd, check=True)  
    video_path = os.path.join(output_path, "video.mp4")  
    assert os.path.exists(video_path)  
    return video_path
```

PHASE 5: Evaluation & Final Deliverables

Goal: Quantitative and qualitative evaluation, final report. **Duration:** Week 8

Task 5.1: Quantitative Metrics (`src/evaluation/metrics.py`)

Implement the following metrics comparing rendered views of the edited scene against ground truth or baseline:

Metric	Library	Purpose
PSNR	skimage.metrics.peak_signal_noise_ratio	Pixel-level reconstruction quality
SSIM	skimage.metrics.structural_similarity	Structural similarity
LPIPS	lpips (pip package)	Perceptual similarity
FPS	Custom timer	Real-time rendering performance

Task 5.2: Demo Video Generation

Write a script that produces a polished demo video showing:

- 1. The original reconstructed scene (orbit camera).
- 2. Mode A: Text prompt → object appears → physics simulation plays.
- 3. Mode B: Text prompt → object highlighted → physics effect applied.

Use `ffmpeg` to concatenate clips with text overlays.

Task 5.3: Ablation Study

Compare SAM 2 segmentation accuracy against the LERF-based approach from Feature Splatting (if time permits). Measure IoU of segmentation masks on 5 test objects.

3. Global Configuration Schema

The main pipeline configuration file (`configs/pipeline_config.yaml`) should follow this schema:

```
# Scene Configuration
scene:
  scannet_scene_id: "scene0000_00"
  data_root: "data/scannet_raw"
  colmap_output: "data/scannet_colmap"
  model_output: "output/base_scene"
  training_iterations: 30000

# Segmentation Configuration
segmentation:
  grounding_dino_checkpoint: "submodules/Grounded-SAM-
2/gdino_checkpoints/groundingdino_swinb_cogcoor.pth"
  sam2_checkpoint: "submodules/Grounded-SAM-2/checkpoints/sam2.1_hiera_large.pt"
  sam2_config: "submodules/Grounded-SAM-2/sam2/configs/sam2.1/sam2.1_hiera_l.yaml"
  box_threshold: 0.3
  text_threshold: 0.25
  multi_view_count: 4
  consensus_min_votes: 2

# Generation Configuration (Mode A)
generation:
  stable_diffusion_model: "stabilityai/stable-diffusion-2-1-base"
  sds_steps: 500
  guidance_scale: 100.0
  object_scale_factor: 0.1 # Scale relative to scene bounding box

# Physics Configuration
physics:
```

```
n_grid: 100
substep_dt: 4e-4
frame_dt: 0.01
frame_num: 120
gravity: -9.8

# Evaluation
evaluation:
  test_views: [0, 5, 10, 15, 20]
  lpips_net: "alex"
```

4. Critical Implementation Notes for Cursor AI

4.1 VRAM Management Strategy

The four major models cannot coexist in GPU memory simultaneously on a typical 24GB GPU. Cursor AI must implement a **sequential loading/unloading** pattern:

```
# Example pattern in pipeline.py
def run_pipeline(config):
    # Phase 1: Train 3DGS (uses ~8GB VRAM)
    train_scene(...)
    torch.cuda.empty_cache()

    # Phase 2: Segmentation (SAM2 + GDINO uses ~12GB VRAM)
    indices = run_segmentation(...)
    del sam2_model, gdino_model
    torch.cuda.empty_cache()
    gc.collect()

    # Phase 3: SDS Generation (SD uses ~10GB VRAM)
    object_ply = generate_object(...)
    del sd_pipeline
    torch.cuda.empty_cache()
    gc.collect()

    # Phase 4: Physics (MPM uses ~6GB VRAM)
    run_simulation(...)
```

4.2 PLY Compatibility Between Submodules

PhysGaussian uses its own fork of `gaussian-splatting` as a git submodule. The PLY format may differ slightly from the main `gaussian-splatting` repo (e.g., different SH coefficient counts). Before merging PLY files, Cursor AI must:

1. Load both the base scene PLY and the PhysGaussian-expected PLY format.
2. Compare vertex property schemas.
3. If they differ, write a conversion function that pads or truncates SH coefficients.

4.3 Coordinate System Alignment

DreamGaussian generates objects in a normalized coordinate space (roughly $[-1, 1]^3$), while ScanNet scenes use metric coordinates. The `object_rescaler.py` must:

1. Compute the bounding box of the base scene.
2. Scale the generated object to a physically plausible size (configurable via `object_scale_factor`).
3. Apply the same rotation/translation that PhysGaussian expects (scene must be axis-aligned).

4.4 PhysGaussian Scene Preparation

PhysGaussian requires the scene to be axis-aligned (e.g., floor parallel to xy-plane). ScanNet scenes may have arbitrary orientations. Cursor AI must:

1. Detect the dominant floor plane using RANSAC on the Gaussian positions.
2. Compute a rotation matrix that aligns this plane with z=0.
3. Store this rotation in the PhysGaussian config JSON (`rotation_degree` , `rotation_axis`).

5. Testing & Validation Checklist

Each phase has specific pass/fail criteria. Do not proceed to the next phase until all criteria are met.

Phase	Test	Pass Criteria
1	setup.sh completes without errors	All imports succeed
1	COLMAP conversion	<code>pycolmap.Reconstruction()</code> loads without error
1	3DGS training	<code>point_cloud.ply</code> exists and has > 100K vertices
1	View rendering	Rendered images are visually recognizable
2	Text-to-mask	Mask overlaid on image correctly highlights the target object
2	Mask-to-Gaussians	Visualize selected Gaussians in 3D; they should form the target object shape
3	SDS generation	Generated object is recognizable from the text prompt
3	Placement	Merged PLY renders correctly; object sits on surface, not floating or clipping
4	Physics simulation	10-frame test runs without OOM; object deforms plausibly
4	Full simulation	120-frame video shows smooth physics animation
5	Metrics	PSNR/SSIM/LPIPS values are computed and saved to CSV

6. Deliverables Summary

Upon completion, the project should produce the following artifacts:

Deliverable	Format	Description
Working pipeline code	Python	End-to-end Mode A and Mode B
Demo video (Mode A)	.mp4	Text → generate object → place → physics
Demo video (Mode B)	.mp4	Text → select object → apply physics effect
Quantitative results	.csv	PSNR, SSIM, LPIPS, FPS per test view
Technical report	.pdf	Method description, results, ablation
Config files	.yaml + .json	Reproducible experiment settings